Kotlin Backend Developer.Professional

otus.ru

REC **Проверить, идет ли запись**

Меня хорошо видно && слышно?

Расширенные возможности Gradle

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS-Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multiplatform	Interop	Gradle
Модуль 3: Подготовка к разработке	Маркетинг аналитика	Security	Архитектура	DevOps		
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin	
Модуль 8: Развитие бизнес-логики проекта	CollabAPI	Машина состояний	Auth	Основы ML	ML в бэкенде	
Модуль 9: Проектная работа	Обзор экосист.	Консуль-тация	Защита			

Команда курса

Окатов Сергей
автор программы, руководитель курса

- кандидат физ.-мат. наук
- 15 лет научной работы в университетах РФ, Европы, США
- 10 лет комм. разработки в стартапах и корпорациях (Сбер, Ланит) в качестве руководителя разработки, архитектора
- Специализация
  - Котлин
  - Большие данные
  - Распределенные системы
  - Архитектура

План вебинара

- Элементы Gradle-проекта
- Структура проекта
- Gradle submodules vs composite builds
- Контроль версий и toml
- Плагины Gradle
- Рефлексия

Цели вебинара

После занятия вы:

1. Сформировать инфраструктуру выпускного проекта
2. Работать с элементами Gradle
3. Создавать и использовать плагины Gradle

Смысл

Зачем вам это уметь

1. Понимать и использовать лучшие практики управления Gradle-проектом
2. Иметь общее представление о возможностях Gradle
3. Знать инструменты DRY, снижать дублирование в инфраструктуре

Общая информация

О вебинаре

1. Это обзорный курс
2. Нет цели изучить весь Gradle
3. Не затронем некоторые темы
   - Конфигурации
   - Артефакты
   - Публикация артефактов
1. Вскользь
   - Взаимодействие модулей
1. По всем дополнительным вопросам - документация и StackOverflow

Элементы Gradle

Элементы Gradle

- Tasks
- Configuration
- Plugins
- Project
- SourceSet

https://docs.gradle.org/current/dsl/index.html

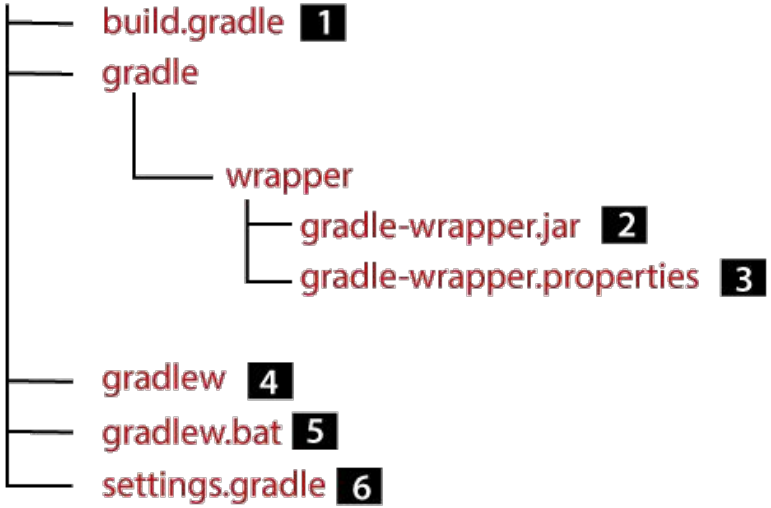
Core types

Listed below are some of the central types which are used in Gradle scripts:

Type	Description
`Project`	This interface is the main API you use to interact with Gradle from your build file. From a `Project`, you have programmatic access to all of Gradle's features.
`Task`	A `Task` represents a single atomic piece of work for a build, such as compiling classes or generating javadoc.
`Gradle`	Represents an invocation of Gradle.
`Settings`	Declares the configuration required to instantiate and configure the hierarchy of `Project` instances which are to participate in a build.
`IncludedBuild`	A build that is included in the composite.
`ProjectLayout`	Provides access to several important locations for a project.

Структура Gradle

1. Скрипт сборки
2. Упаковщик Gradle
   a. Скачивает Gradle нужной версии
3. Настройки упаковщика
   a. Указывает версию и параметры Gradle
4. Unix-скрипт для задач Gradle
   a. Запускает упаковщик
   b. Выполняет задачи
5. Windows-скрипт для задач Gradle
6. Скрипт настроек проекта



settings.gradle.kts

- Один на весь проект
- Задает предварительные настройки проекта
- Читается первым
- Отражает жизненный цикл сборки

Содержит:

1. Имя проекта
2. Настройки вычислений зависимостей
3. Композитные сборки
4. Подпроекты
5. Настройки плагинов

```
rootProject.name = "ad"

dependencyResolutionManagement {
   versionCatalogs {
       create("libsbe") {
           from(files("./gradle/libs.be.versions.toml"))
       }
       create("libsfe") {
           from(files("./gradle/libs.fe.versions.toml"))
       }
   }
}

includeBuild("ad-docs")
includeBuild("ad-back")
includeBuild("ad-frontlib")

include(":ad-fe-app-andr")
include(":ad-fe-app-js")
include(":ad-fe-app-desktop")

pluginManagement {
   repositories {
       gradlePluginPortal()
       google()
       maven("https://maven.pkg.jetbrains.space/public/p/compose/dev")
       maven("https://maven.google.com/")
   }
}
```

build.gradle.kts

Свой для каждого подпроекта
Правила сборки подпроекта

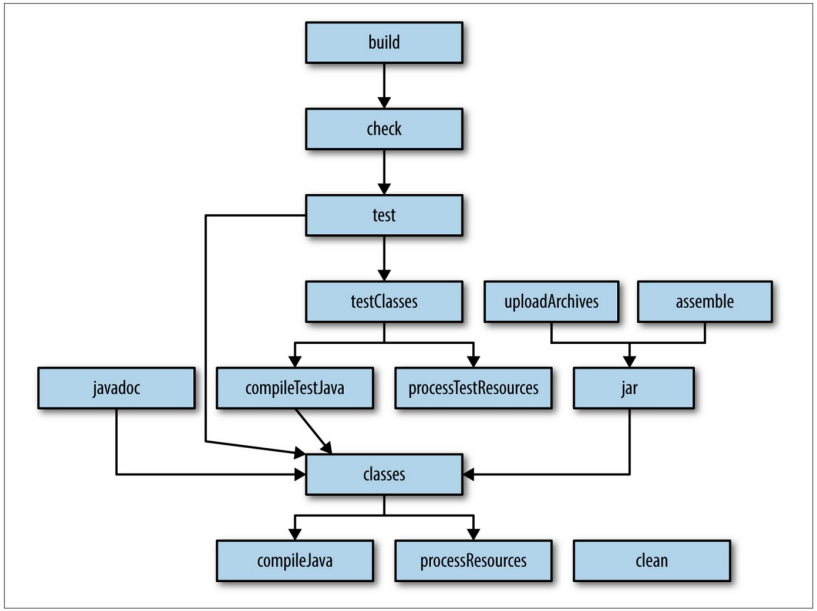
Содержит:

1. Подключение плагинов
2. Настройки проекта/подпроекта (группа, версия и пр.)
3. Репозитории
4. Настройки конкретных плагинов
5. Таски
6. Конфигурации
7. Настройки артефактов и публикации

```
plugins {
   id("com.crowdproj.plugin.autoversion") version "0.0.5"
   alias(libsfe.plugins.kotlin.jvm) apply false
}

group = "com.crowdproj.ad"
version = "1.0-SNAPSHOT"

repositories {
   mavenCentral()
}

autoversion {
   shoudIncrement.set(false)
}

subprojects {
   group = rootProject.group
   version = rootProject.version
}

allprojects {
   repositories {
       google()
       mavenCentral()
       maven("https://maven.pkg.jetbrains.space/public/p/compose/dev")
   }
}
```

Tasks

1. Выполняют основные задачи сборки
2. Связаны между собой
3. Образуют ациклический граф
4. Оперируют файлами:
   a. inputs
   b. output
5. Инкрементальная сборка => контроль изменений inputs
6. DSL …



Tasks

1. Выполняют основные задачи сборки
2. Связаны между собой
3. Образуют ациклический граф
4. Оперируют файлами:
   a. inputs
   b. output
5. Инкрементальная сборка => контроль изменений inputs
6. DSL, жизненный цикл:
   a. настройка задач
   b. после настройки задач
   c. выполнение задач
      i. вначале
      ii. вконце

```
tasks {
   create("myTask") {
       println("Configuration stage")
       doFirst { println("At task starts") }
       doLast { println("At task ends") }
   }
}

afterEvaluate {
   tasks {
       create("myOtherTask") {
           println("After other tasks initialized")
       }

       forEach { println("TASK $it") }
   }
}
```

LIVE

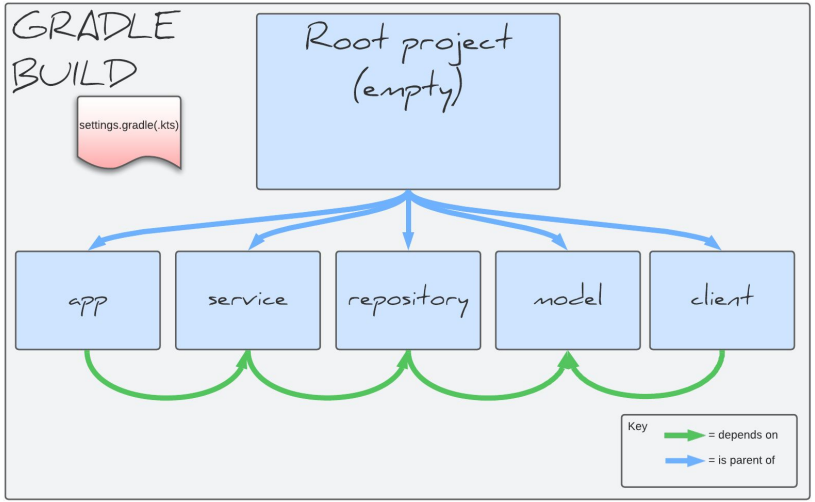
управление тасками

Подпроекты и композитные сборки

Подпроекты

Особенности

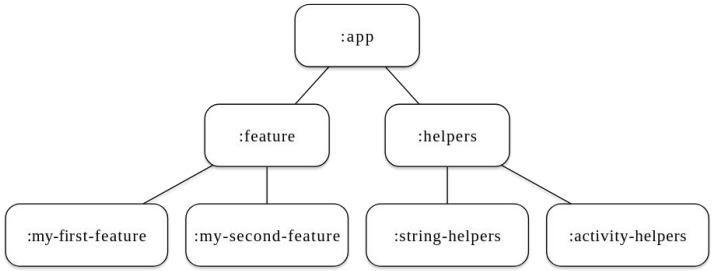
1. В терминах Maven - модули
2. Позволяют структурировать проект
3. Изолировать друг от друга логические блоки кода
4. Контролировать зависимости
5. Выстраивать иерархию подпроектов
6. Доступ через *implementation(project(“:sub”))*



Подпроекты

Иерархия

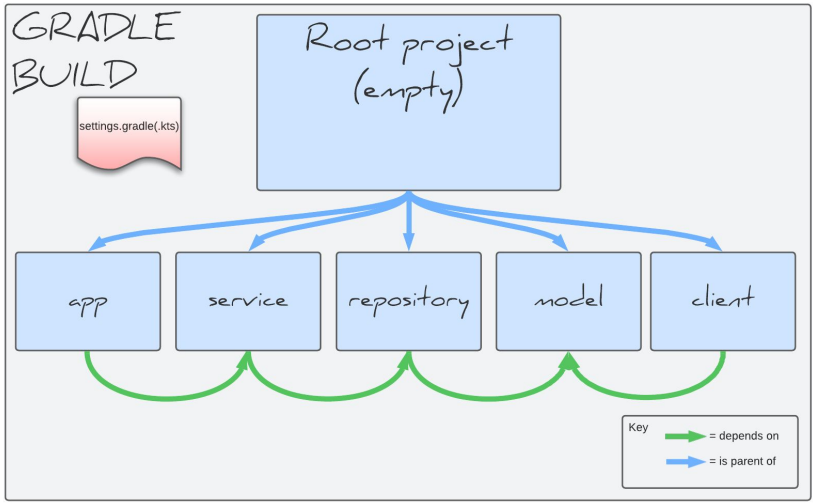
1. Любая вложенность
2. Доступ через project(“:sub1:sub2:sub3”)
3. Или явное указание пути




```
// settings.gradle.kts

enableFeaturePreview("TYPESAFE_PROJECT_ACCESSORS")

// build.gradle.kts

implementation(projects.m2l5Gradle.sub1.ssub1)
implementation(projects.m2l5Gradle.sub1.ssub2)
```

```
include(
    ":m2l5-gradle:sub1:ssub1",
    ":m2l5-gradle:sub1:ssub2"
)

include(":m2l5-gradle-sub2")
project(":m2l5-gradle-sub2").apply {
   projectDir = file("m2l5-gradle/sub2")
   name = "m2l5-custom-sub2"
}
```

Подпроекты

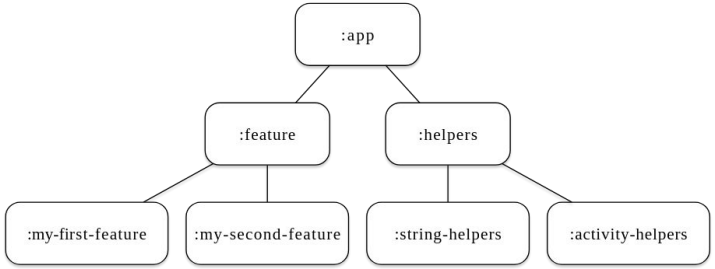


Возможности

1. Позволяют выстраивать единую политику в рамках всего проекта
   a. версии
   b. зависимости
   c. плагины

Не позволяют

1. Разные версии плагинов в рамках одного проекта
2. Замещать сторонние библиотеки
3. Все подпроекты указываются в корневом settings.gradle.kts (“плоская иерархия”)

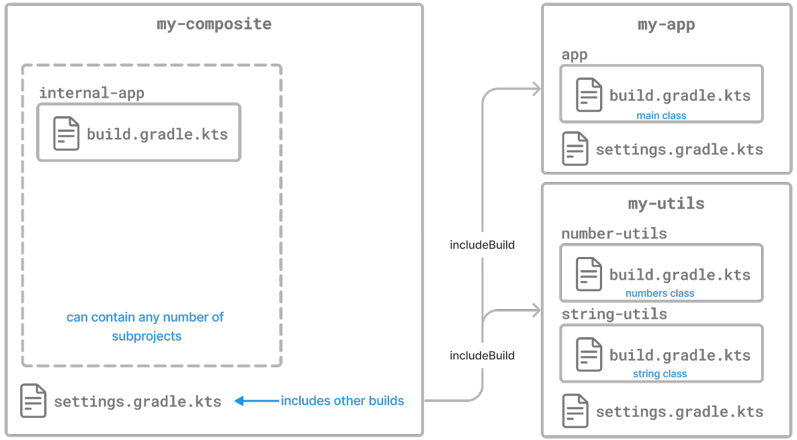


Композитные сборки

1. Полный набор Gradle-файлов, включая settings, properties и пр.
2. Полная компиляция с имитацией публикации в мавен-реестре
3. Доступ через *implementation("group-id:module")*

Используются:

1. Для сборки плагинов
2. Для работы с внешними библиотеками
3. Для изоляции от корня проекта
4. Для существенно разных проектов
   a. фронт
   b. бэк
   c. приемочные тесты



Композитные сборки

Подключение

```
settings.gradle.kts:

includeBuild("my-composite-module")

implementation("my.group.id:my-composite-module")
```

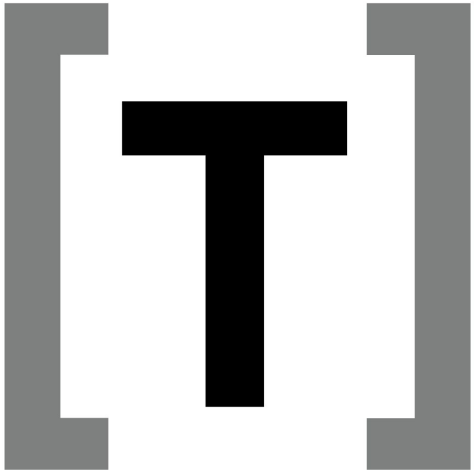
LIVE

подпроекты, композитные сборки

Контроль зависимостей

Типы контроля зависимостей

1. Ext - шаринг переменных, в том числе версий зависимостей
2. gradle.properties
   a. settings: val version by setting
   b. build: val version by project
3. libs.version.toml



Toml

Контролирует

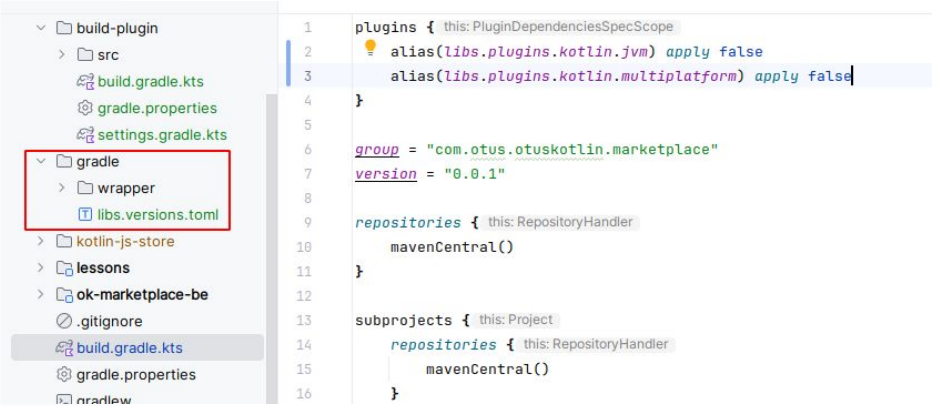
1. Номера версий
2. Зависимости библиотек
3. Плагины

```
[versions]
kotlin = "1.9.22"
kotlinx-datetime = "0.5.0"

[libraries]
kotlinx-datetime = {module = "org.jetbrains.kotlinx:kotlinx-datetime", version.ref = "kotlinx-datetime"}

[plugins]
kotlin-multiplatform = {id = "org.jetbrains.kotlin.multiplatform", version.ref = "kotlin"}
kotlin-jvm = {id = "org.jetbrains.kotlin.jvm", version.ref = "kotlin"}

[bundles]
kotest = ["kotest-junit5", "kotest-core", "kotest-datatest", "kotest-property"]
```

Подключение в проектах

1. Если в проекте есть /gradle, то подключено автоматом
2. Если используем чужой libs.versions.toml, то в settings.gradle.kts

```
dependencyResolutionManagement {
    versionCatalogs {
        create("libs") {
            from(files("../gradle/libs.versions.toml"))
        }
    }
}
```



Использование в проектах

```
1.  alias(libs.plugins.my.plugin)
2.  implementation(libs.my.library)
3.  val version = libs.versions.my.get()
```

```
plugins {
   alias(libs.plugins.kotlin.jvm)
}
val kotlinVersion = libs.versions.kotlin.get()
dependencies {
   implementation(libs.kotlinx.datetime)
}
```

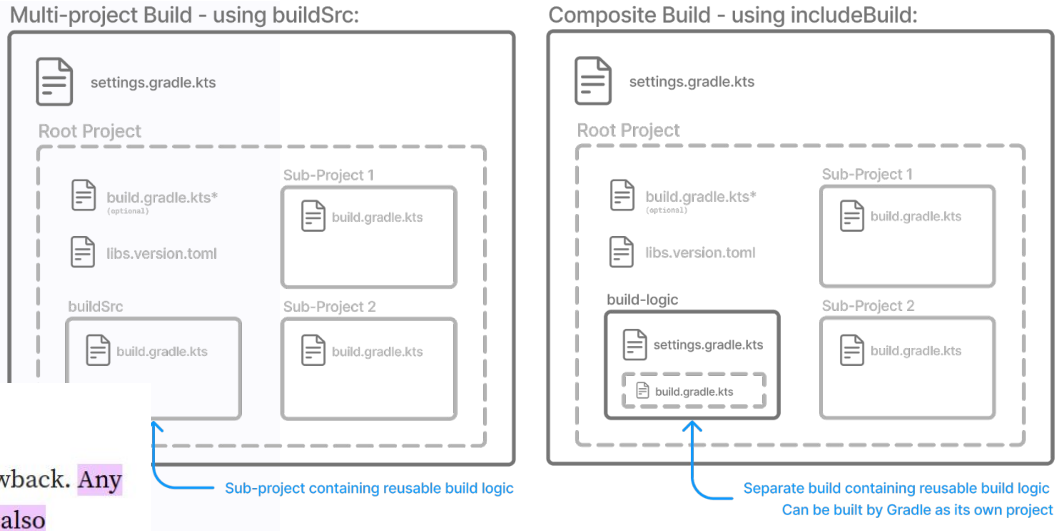
LIVE

Работа с toml-файлом

Плагины Gradle

Разделяемая логика сборки

1. `buildSrc`
   a. собирается каждый раз
2. `Plugin (includeBuild)`
   a. собирается при изменениях



Drawback of `buildSrc`

Unfortunately, all these cool features come with one massive drawback. Any change inside `buildSrc` completely invalidates the build cache. It also invalidates remote build cache, in instances when you are using it. Whilst it's not really a problem for small projects, big ones with hundreds of modules are affected badly. On the other hand, changes in Gradle script files don't invalidate the cache but merely invalidate some tasks.

Плагин

1. композитный подпроект
2. Добавляется
   a. НЕ в корневом settings
   b. В проектном settings.gradle.kts
   c. НЕ в основном блоке
   d. В `pluginManagement`

```
pluginManagement {
   includeBuild("../build-plugin")
   plugins {
       id("build-jvm") apply false
       id("build-kmp") apply false
   }
   repositories {
       mavenCentral()
       gradlePluginPortal()
   }
}
```

Плагин: настройка проекта плагина

Регистрация плагина в

/plugin/build.gradle.kts

```
plugins {
    `kotlin-dsl`
}

gradlePlugin {
    plugins {
        register("build-jvm") {
            id = "build-jvm"
            implementationClass = "ru.otus.otuskotlin.marketplace.plugin.BuildPluginJvm"
        }
    }
}
```

Плагин: реализация

В apply определяем все настройки плагина:

1. Подгрузка связанных плагинов
2. Задаем расширения
3. Читаем настройки из прикладного скрипта

```kotlin
internal class BuildPluginJvm : Plugin<Project> {

    override fun apply(project: Project) = with(project) {
        pluginManager.apply("org.jetbrains.kotlin.jvm")
        group = rootProject.group
        version = rootProject.version
    }
}
```

LIVE

Реализация плагина

Рефлексия и подведение итогов

Цели вебинара

Проверка достижения

1. Сформировать инфраструктуру выпускного проекта
2. Работать с элементами Gradle
3. Создавать и использовать плагины Gradle

Тезисы

1.	Gradle содержит много элементов, наиболее значимый - Task. Таски формируют ациклический граф
2.	Иерархия проекта формируется через include (подпроекты) и includeBuild (композитные сборки)
3.	Контроль зависимостей в Gradle с помощью ext, gradle.property, libs.versions.toml
4.	Разделяемая логика сборки через builsSrc или плагины

Слайд с домашним заданием

1. создать три композитных проекта: (1) учебные модули; (2) плагин; (3) проектные модули
2. создать плагины Gradle для сборки подпроектов (1) на базе jvm; (2) на базе multiplatform
3. создать файл libs.versions.toml и зацепить за него версии зависимостей в плагинах

Рефлексия

Остались ли у вас насущные вопросы по теме?

Какой аспект темы показался самым сложным?

2 мин

Пишем в чат или говорим голосом

Список материалов для изучения

1. Kotlin: https://kotlinlang.org/
2. Gradle: https://gradle.org/
3. Учебный проект: https://github.com/otuskotlin/202505-ok-marketplace

Следующий вебинар

Модуль 3: Подготовка к разработке

Маркетинг и аналитика проекта // ДЗ

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание! Приходите на следующие вебинары